

Statement vs PreparedStatement vs CallableStatement

09 August 2026 11:40

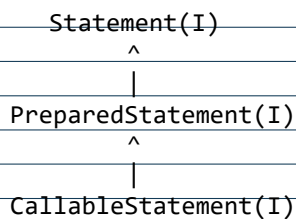
1. We can use normal Statement to execute multiple queries:

- i. `st.executeQuery(query1);`
- ii. `st.executeQuery(query2);`
- iii. `st.executeQuery(query3);`

i.e., if we want to work with multiple queries then we should go for Statement object.

2. If we want to with only one query, but should be executed multiple times then we should go for PreparedStatement.

3. If we want to work with stored procedure and functions then we should go for CallableStatement



`Statement st = con.createStatement();`

↑ this statement can execute any number of queries or multiple queries.

`PreparedStatement pst = con.prepareStatement(query);`

`pst.executeUpdate();` Advantage: the query will be

`pst.executeUpdate();` compiled only once. So, performance will improve.

`CallableStatement st = con.prepareCall("{ call getEmpInfo(??) }");`
`st.execute();`



Need of Batch Update

- o Statement Batch updates.
- o PreparedStatement Batch updates.
- o CallableStatement Batch Updates.
- o Batch Updates and Transactions.

Batch Update

A batch update is a batch of updates grouped together and sent to the database in one "batch" rather than sending the updates one by one. So there is less network traffic involved in sending one batch of updates. (only one round trip) And the database might

be able to execute some of the updates in parallel. The speed up compared to the executing the updates one by one. You can batch SQL insert, update and deletes. It does not make sense to batch select statements.

There are three ways to execute batch updates:

1. Using a Statement
2. Using a PreparedStatement
3. Using a CallableStatement

Statement Batch updates

You can use a statement object to execute batch updates. You do so using the `addBatch()` and `executeBatch()` methods. Here is an example:

```
Statement st = con.createStatement();
st.addBatch("UPDATE employee set ename = 'Vijay' where emp_id = 'E020' ");
st.addBatch("UPDATE employee set ename = 'Ravi' where emp_id = 'E030' ");

int[] recordsAffected = st.executeBatch(); //
//Submits a batch of commands to the database for execution and if all commands execute
//successfully, returns an array of update counts.
```

From <<https://docs.oracle.com/javase/8/docs/api/java/sql/Statement.html>>

```
st.close();
```

PreparedStatement Batch Updates:

You can also use a PreparedStatement object to execute batch updates. The prepared statement enables you to reuse the same SQL statement and just insert a new parameters into it for each update to execute. For example:

```
String sql = "update employee set ename=?, basic_salary = ? where emp_id = ?";
```

```
PreparedStatement pst = con.prepareStatement(sql);
pst.setString(1, "Ram");
pst.setFloat(2, 2000.0);
pst.addBatch();
```

```
pst.setString(1 "Bharat");
pst.setFloat(2, 3000.0);
pst.addBatch();
```

```
int[] affectedRecords = pst.executeBatch();
```

```
pst.close();
```

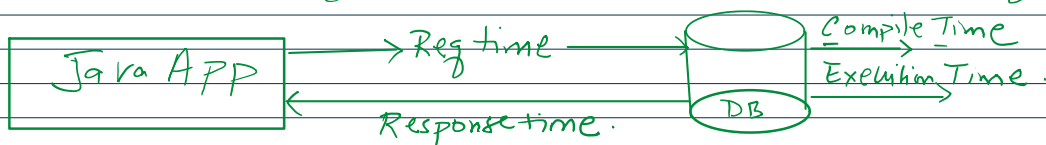
3. CallableStatement Batch Updates: It works similar to PreparedStatement Batch Updates

It is important to keep in mind that each update added to. A statement or prepared statement is executed separately by the database. That means if some of them may succeed before one of them fails, all the statements that have succeeded are now committed to the database, but the rest of the updates may not be. This can result in an inconsistent data in the database. To avoid this, you can execute the batch update inside a transaction. When executed inside a transaction, you can make sure that either all updates are executed or none. Any successful updates can be rolled back in case of update fails.

Need of batch updates

1. Statement:- When we submit multiple SQL queries to the database

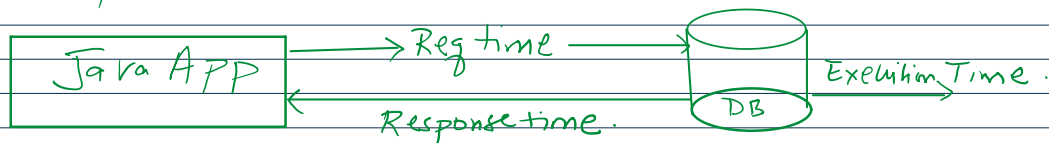
one by one then lot of time wasted in request and response.



$$\begin{aligned}\text{Time / query} &= \text{Reg.time} + \text{C.T.} + \text{E.T.} + \text{Resp.T.} \\ &= 1\text{ms} + 1\text{ms} + 1\text{ms} + 1\text{ms} = 4\text{ms}.\end{aligned}$$

$$\text{Time / 1000 query} = (4 \times 1000) = 4000\text{ ms}$$

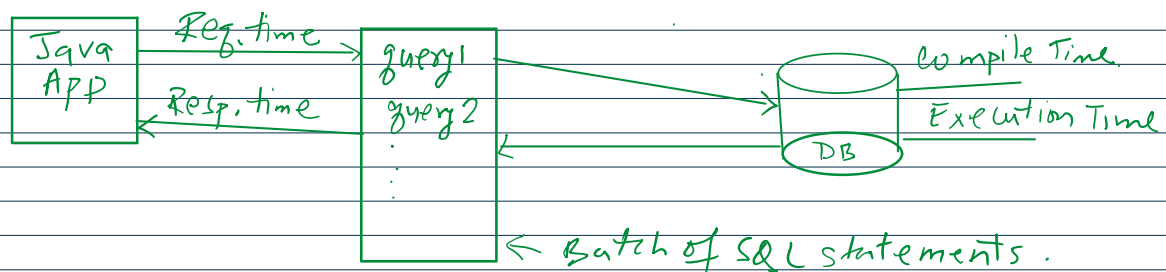
2. Prepared Statement:



$$\begin{aligned}\text{Time / query} &= \text{Reg.time} + \text{E.T.} + \text{Resp.T.} \\ &= 1\text{ms} + 1\text{ms} + 1\text{ms} = 3\text{ms}.\end{aligned}$$

$$\text{Time / 1000 query} = (3 \times 1000) = 3000\text{ ms}$$

To overcome these problems, we should go for batch update. We can group all related SQL queries in a single batch and we can send that batch at a time to database.



$$\begin{aligned}\text{Time per 1000 queries} &= \text{R.T} + 1000 \times \text{C.T.} + 1000 \times \text{ET} + \text{Resp.T.} \\ &= 1 + 1000 \times 1\text{ms} + 1000 \times 1 + 1\text{ms} = 2002\text{ ms}.\end{aligned}$$

Hence the main advantage of batch updates are:

1. We can reduce network traffic.
2. We can improve performance.

We can implement batch updates by using the following

We can implement batch updates by using the following two methods:

1. `public void addBatch (String sqlQuery)`

2. `int[] executeBatch()` ← to execute batch of sql queries.

Before the update of basic salary by 10%

EMP_ID	NAME	DEPT_ID	BASIC_SALARY
1 E002	Naveen Singh	D001	67800
2 E003	Sangita Mishra	D004	45700
3 E004	Ajay Dewan	D002	34000
4 E005	Bali Agarwal	D002	35600
5 E006	Gargi	IT	70000
6 E007	Garvit	D004	45000
7 E008	Yashika	D003	46500
8 E009	Amar	D003	43800
9 E010	Frank	IT	67000
10 E011	Rickard	Hardware	70000
11 E012	Roberto	Hardware	70100
12 E013	Sakshi	IT	67800
13 E014	Eeshan	D001	45600
14 E015	Rajveer	D002	71200
15 E016	Jagdish	D003	46800
16 E001	Akshay	D001	50000

```
package MyJDBCOne;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
public class ConnectionObject {
    private static String dbUrl = "jdbc:oracle:thin:@//localhost:1521/FREE";
    private static String username = "sys as sysdba";
    private static String password = "Calcutta1983";
    @SuppressWarnings("finally")
    public static Connection getConnection(){
        Connection con=null;
        try {
            con = DriverManager.getConnection(dbUrl, username, password);
        } catch (SQLException e) {
            e.printStackTrace();
        }
        finally{
            return con;
        }
    }
}
```

```
package MyJDBCOne;
import java.sql.Connection;
import java.sql.SQLException;
import java.sql.Statement;
public class App {
    Connection con = ConnectionObject.getConnection();
    public static void main(String[] args) {
        App ap = new App();
    }
}
```

```

        ap.batchUpdateUsingStatement();
    }
    public void batchUpdateUsingStatement(){
        int result[] = null;
        try {
            Statement stmt = con.createStatement();
            con.setAutoCommit(false);
            String SQL = "UPDATE employee "+
                "SET basic_salary = basic_salary + basic_salary*0.1 "+
                "WHERE basic_salary <= 50000 ";
            stmt.addBatch(SQL);
            result = stmt.executeBatch();
            con.commit();
        } catch (SQLException e) {

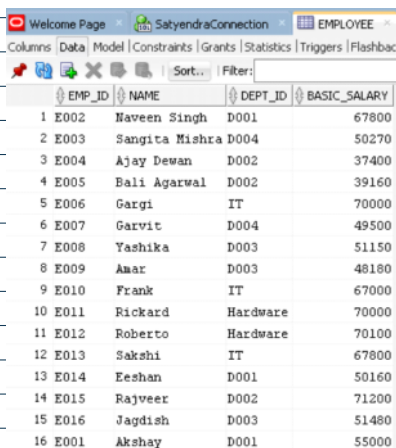
            e.printStackTrace();
        }
        finally{
            if(result!=null){
                for (int r: result) {
                    System.out.println(r);
                }
            }
        }
    }
}
}

```

After update:

EMP_ID	NAME	DEPT_ID	BASIC_SALARY
1 E002	Naveen Singh	D001	67800
2 E003	Sangita Mishra	D004	50270
3 E004	Ajay Dewan	D002	37400
4 E005	Bali Agarwal	D002	39160
5 E006	Gargi	IT	70000
6 E007	Garvit	D004	49500
7 E008	Yashika	D003	51150
8 E009	Amar	D003	48180
9 E010	Frank	IT	67000
10 E011	Rickard	Hardware	70000
11 E012	Roberto	Hardware	70100
12 E013	Sakshi	IT	67800
13 E014	Eeshan	D001	50160
14 E015	Rajveer	D002	71200
15 E016	Jagdish	D003	51480
16 E001	Akshay	D001	55000

Before insertion:



EMP_ID	NAME	DEPT_ID	BASIC_SALARY
1 E002	Naveen Singh	D001	67800
2 E003	Sangita Mishra	D004	50270
3 E004	Ajay Dewan	D002	37400
4 E005	Bali Agarwal	D002	39160
5 E006	Gargi	IT	70000
6 E007	Garvit	D004	49500
7 E008	Yashika	D003	51150
8 E009	Amar	D003	48180
9 E010	Frank	IT	67000
10 E011	Rickard	Hardware	70000
11 E012	Roberto	Hardware	70100
12 E013	Sakshi	IT	67800
13 E014	Eeshan	D001	50160
14 E015	Rajveer	D002	71200
15 E016	Jagdish	D003	51480
16 E001	Akshay	D001	55000

```

package MyJDBCOne;
import java.sql.Connection;

```

```

import java.sql.PreparedStatement;
import java.sql.SQLException;
import java.sql.Statement;
public class App {
    Connection con = ConnectionObject.getConnection();
    public static void main(String[] args) {
        App ap = new App();
        // ap.batchUpdateUsingStatement();
        ap.batchUpdateUsingPreparedStatement();
    }
    public void batchUpdateUsingStatement(){
        int result[] = null;
        try {
            Statement stmt = con.createStatement();
            con.setAutoCommit(false);
            String SQL = "UPDATE employee "+
                "SET basic_salary = basic_salary + basic_salary*0.1 "+
                "WHERE basic_salary <= 50000 ";
            stmt.addBatch(SQL);
            result = stmt.executeBatch();
            con.commit();
        } catch (SQLException e) {

            e.printStackTrace();
        }
        finally{
            if(result!=null){
                for (int r: result) {
                    System.out.println(r);
                }
            }
        }
    }
    public void batchUpdateUsingPreparedStatement(){
        int result[] = null;
        try {
            String SQL = "INSERT INTO employee "+
                "(emp_id, name, dept_id, basic_salary )" +
                "VALUES(?,?,?,?)";

            PreparedStatement stmt = con.prepareStatement(SQL);
            con.setAutoCommit(false);
            //employee - 1
            stmt.setString(1, "E017");
            stmt.setString(2, "Tamim");
            stmt.setString(3, "IT");
            stmt.setInt(4, 45900);
            stmt.addBatch();
            //employee - 2
            stmt.setString(1, "E018");
            stmt.setString(2, "Rishi");
            stmt.setString(3, "D001");
            stmt.setInt(4, 35900);
            stmt.addBatch();
            //employee - 3
            stmt.setString(1, "E018");
            stmt.setString(2, "Rashi");
            stmt.setString(3, "IT");
            stmt.setInt(4, 59900);
            stmt.addBatch();
            result = stmt.executeBatch();
            con.commit();
        } catch (SQLException e) {

```

```

        e.printStackTrace();
    }
    finally{
        if(result!=null){
            for (int r: result) {
                System.out.println(r);
            }
        }
    }
}
}
}

```

1
1
1
PS D: } ← 3 records are updated, in a batch.

After insertion:

EMP_ID	NAME	DEPT_ID	BASIC_SALARY
1 E017	Tania	IT	45900
2 E002	Naveen Singh	D001	67800
3 E003	Sangita Mishra	D004	50270
4 E004	Ajay Dewan	D002	37400
5 E005	Bali Agarwal	D002	39160
6 E006	Gargi	IT	70000
7 E007	Garvit	D004	49500
8 E008	Yashika	D003	51150
9 E009	Amar	D003	48180
10 E010	Frank	IT	67000
11 E011	Rickard	Hardware	70000
12 E012	Roberto	Hardware	70100
13 E013	Sakshi	IT	67800
14 E014	Eeshan	D001	50160
15 E015	Rajveer	D002	71200
16 E016	Jagdish	D003	51480
17 E001	Akshay	D001	55000
18 E018	Rishi	D001	35900
19 E018	Rashi	IT	59900

explains.

1. executeQuery() }
 2. executeUpdate() }
 3. execute() }
 4. executeBatch() }
 int[]